Exercício Extra

Aula 06 — Montar um Sistema BLE e Demonstrá-lo

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026–2027

Catarina Silva

ESTGA, Universidade de Aveiro

Enquadramento

Neste exercício o objetivo é reunir tudo num sistema que funcione de ponta a ponta e, sobretudo, usá-lo para *observar e medir* o comportamento do BLE.

O sistema é um posto de monitorização de uma sala. Um nó com Arduino e módulo HM-11 mede temperatura e humidade e é comandado por BLE. Um Raspberry Pi liga-se ao nó, recebe as leituras, regista-as e apresenta-as num painel. O que se avalia no fim não é o painel em si, mas o que o grupo consegue demonstrar e explicar sobre a tecnologia de comunicação que o sustenta.

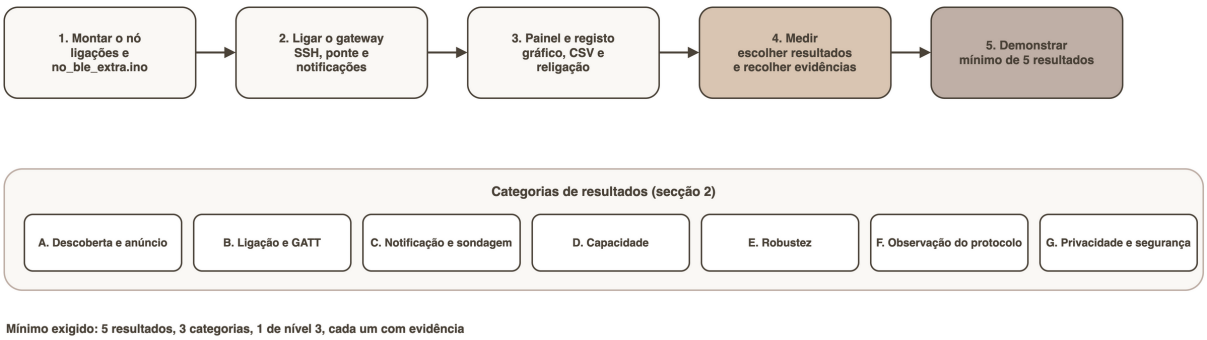


Figura 1: *Etapas do exercício*

Material

- Raspberry Pi com cartão preparado, com SSH ativo, ligado à rede local do laboratório, com BlueZ, bluepy e Node-RED com o pacote `@flowfuse/node-red-dashboard`.
- Computador pessoal, para o acesso por SSH e para ver o painel.
- Arduino com shield HM-11. O DHT11 e o LED, ligado ao pino 8 com resistência de 220 Ω, são opcionais (subsecção “Sem todos os sensores”).
- Telemóvel com a aplicação *nRF Connect* (ou equivalente), como segundo cliente.
- Pasta `codigo/`, com `ponte_ble_tcp.py`, da Aula 05, e ainda `no_ble_extra.ino`, `ponte_estendida.py`, `ponte_at.ino`, `varredor.py` e `medidor_ble.py`.

Cada grupo precisa de um nó, com Arduino e HM-11, e de um Raspberry Pi. Se faltar um deles, dois grupos partilham o material e alternam, porque o HM-11 aceita um cliente de cada vez. Enquanto um grupo mede, o outro prepara o painel, lê o código ou organiza as evidências.

Objetivos de Aprendizagem

- Integrar periférico, gateway e visualização num único sistema BLE funcional;
- Medir grandezas que caracterizam uma ligação BLE (RSSI, latência, cadência, perdas);
- Relacionar o que se observa com os parâmetros do protocolo (intervalo de anúncio, intervalo de ligação, *supervision timeout*, MTU);
- Sustentar cada afirmação com uma evidência: um número, um gráfico ou uma captura.

1. Sistema a Montar

Ligações no Arduino

O shield HM-11 liga-se como na Aula 05. O DHT11 e o LED são acrescentados nos pinos indicados. O LED só deve ser ligado com a resistência em série.

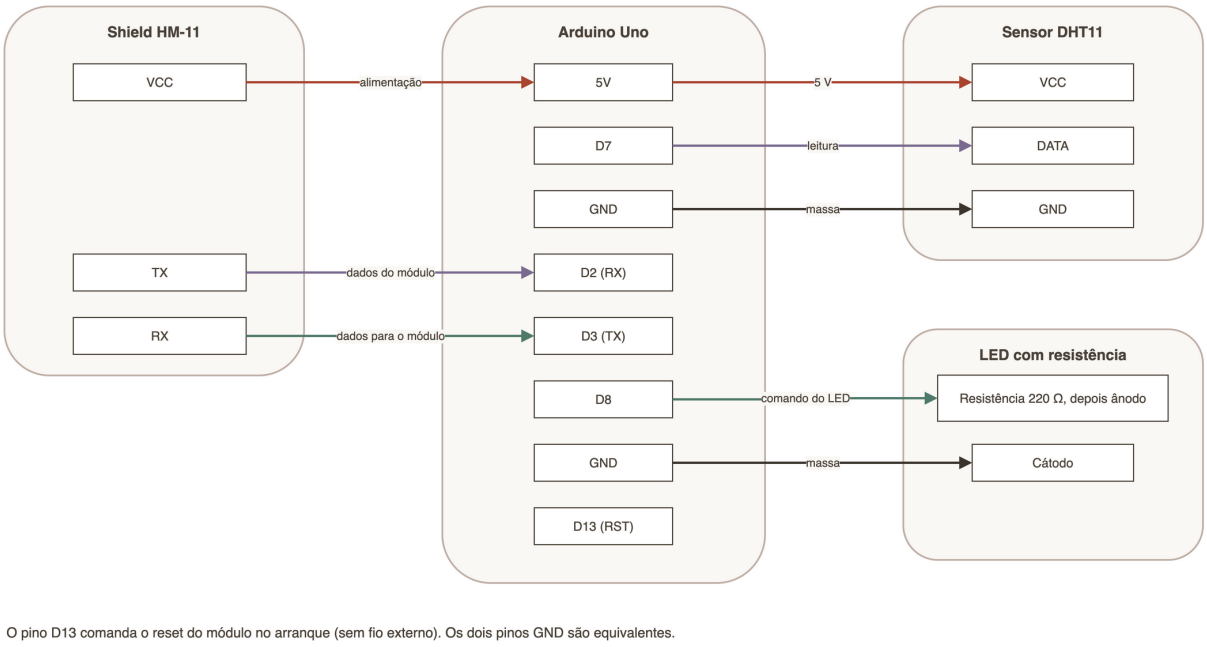


***Figura 2:** Ligações do nó BLE*

Arquitetura

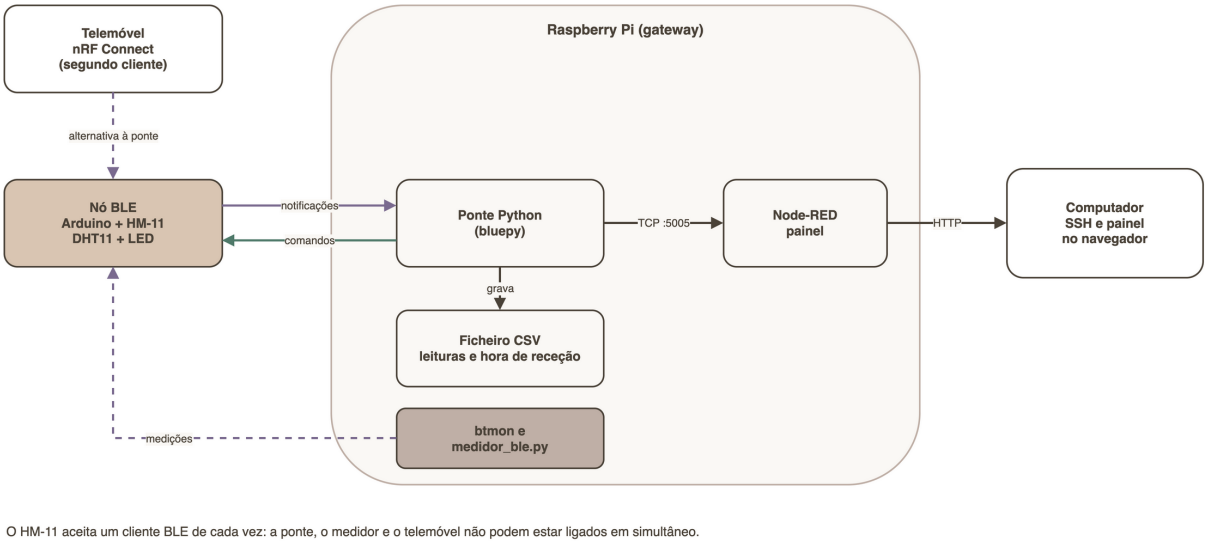


Figura 3: *Arquitetura do sistema e o que corre em cada elemento*

Acesso ao Raspberry Pi

Numa instalação real o gateway fica no local onde estão os sensores, sem monitor nem teclado. O trabalho é por isso feito por SSH a partir do computador pessoal, o que reproduz essa condição e evita um monitor por posto.

1. Obter o endereço atribuído ao Raspberry Pi na rede do laboratório.
2. Abrir a sessão remota e registar a versão do sistema operativo.
3. Confirmar que o adaptador Bluetooth existe e colocá-lo em estado operacional.

```
ssh pi@<ENDERECO_DO_RASPBERRY>
cat /etc/os-release
hciconfig
sudo hciconfig hci0 up
```

Descoberta do nó

Com o nó ligado, executar o varredor. Lista os dispositivos BLE que se anunciam na vizinhança, com o endereço, o tipo de endereço, o RSSI e o nome, e é a forma de obter o endereço do nó.

```
sudo python3 codigo/varredor.py 15
```

O resultado é mais rico do que o do comando `hcitool lescan` da Aula 04, porque inclui o RSSI e o nome extraídos dos dados de anúncio. O RSSI é apresentado em dBm e é sempre negativo. Serve para ordenar dispositivos por proximidade relativa, mas não é uma medida de distância.

A ponte

A ponte liga-se ao nó, ativa as notificações escrevendo no descritor de configuração da característica e reenvia cada notificação recebida para `127.0.0.1:5005`, de onde o Node-RED a consome.

```
sudo python3 codigo/ponte_ble_tcp.py <ENDERECO_DO_NO>
```

Painel Node-RED

1. Arrastar um nó `tcp in`, em modo *Connect to*, para `127.0.0.1:5005`, com entrega de *strings* delimitadas por `\n`.

2. Arrastar um nó `function` com duas saídas e colar o conteúdo seguinte, que lê as posições 1 e 2 porque a posição 0 é o contador:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[1]) },
         { payload: parseFloat(v[2]) } ];
```

3. Associar a primeira saída a um `ui-chart` designado *Temperatura* e a segunda a um `ui-gauge` designado *Humidade relativa*.

4. Premir `Deploy` e abrir o painel.

Fases de realização

O trabalho faz-se em três fases, cada uma sobre a anterior. Para a demonstração basta ter a primeira fase a funcionar.

1. Na primeira fase o nó corre `no_ble_extra.ino`, a ponte da Aula 05 recebe as notificações e o painel Node-RED apresenta pelo menos um gráfico.

2. Na segunda fase a ponte da Aula 05 é substituída pela ponte estendida, que permite enviar comandos ao nó, por exemplo acender e apagar o LED, e grava cada leitura num ficheiro CSV com a hora de receção.

3. Na terceira fase testa-se a religação automática da ponte estendida, desligando o nó e voltando a ligá-lo, e acrescenta-se ao painel um indicador do estado da ligação.

A ponte estendida

A ponte estendida é fornecida pronta. O trabalho consiste em executá-la, lê-la e adaptá-la, por exemplo para mudar o período de envio, alterar o formato das mensagens ou acrescentar um campo ao ficheiro.

```
sudo python3 codigo/ponte_estendida.py <ENDERECO_DO_NO> leituras.csv
sudo python3 codigo/ponte_estendida.py <ENDERECO_DO_NO> leituras.csv --
   sondagem
```

Cada linha escrita no terminal é enviada ao nó como comando. O ficheiro regista a hora de cada mensagem e de cada evento de ligação, com o número acumulado de mensagens perdidas, obtido a partir do contador do nó. Se a ligação cair, a ponte volta a tentar com espera crescente, ativa de novo as notificações e retoma o envio. A opção `-sondagem` pede uma leitura por segundo em vez de usar o envio espontâneo do nó, o que permite comparar os dois modos. O indicador do estado da ligação, no painel, pode apoiar-se num nó `function` que assinala a ausência de leituras durante um certo intervalo.

Comandos do nó

Os comandos são caracteres únicos escritos na característica de dados. Os caracteres `t`, `h` e `X` pedem uma leitura. Os caracteres `1` e `0` acendem e apagam o LED. O caracter `S` ativa o envio espontâneo e `P` desativa-o. Os caracteres `a` a `e` escolhem o período desse envio: 1, 2, 5, 10 ou 30 s. No modo `X` cada mensagem tem a forma `contador,temperatura,humidade`, o que permite detetar mensagens perdidas. Na Aula 05 a mensagem tinha só temperatura e humidade, pelo que o nó `function` do painel, que lia as posições 0 e 1, passa a ler as posições 1 e 2.

Configuração do módulo

Alguns resultados da secção seguinte exigem alterar o nome, o intervalo de anúncio ou a potência de emissão do HM-11. Para isso carregar `ponte_at.ino`, abrir o Monitor Série a 9600 baud sem fim de linha e enviar comandos `AT`, por exemplo `AT+NAME?`, `AT+ADVI?` e `AT+POWE?`. Estes comandos só são aceites quando o módulo não tem nenhuma ligação ativa. Depois de configurar, voltar a carregar `no_ble_extra.ino`.

2. Resultados para Demonstração

Com o sistema a funcionar, cada grupo escolhe os resultados que quer demonstrar. Cada linha das tabelas seguintes descreve um resultado possível e a evidência que o sustenta. O nível indica o esforço relativo: 1 é uma observação direta, 2 exige uma medição repetida e 3 exige uma medição, uma captura ou uma modificação ao sistema.

A. Descoberta e anúncio

Id	Resultado a demonstrar	Evidência	Nível
A1	O nó anuncia-se com um nome escolhido pelo grupo e esse nome aparece no gateway e no telemóvel.	Saída do varredor e ecrã do telemóvel.	1
A2	Decodificação dos dados de anúncio: *flags*, nome, potência de emissão e eventuais dados de fabricante.	Tabela com cada campo, o seu tipo AD e o seu valor.	2
A3	O RSSI diminui com a distância, mas com grande variabilidade.	Gráfico do RSSI em pelo menos quatro distâncias, 30 amostras cada, com média e desvio padrão.	2
A4	Efeito de obstáculos (corpo, parede, gaveta metálica) no RSSI à mesma distância.	Tabela comparativa e conclusão sobre a utilidade do RSSI como medida de distância.	2
A5	O intervalo de anúncio determina quantas vezes o nó é detetado num varrimento.	Contagem de deteções em 10 s para pelo menos três valores de `AT+ADVI`.	3
A6	A potência de emissão altera o RSSI e o alcance.	RSSI à mesma posição para pelo menos dois valores de `AT+POWE`.	3

B. Ligação e GATT

Id	Resultado a demonstrar	Evidência	Nível
B1	O perfil GATT do nó: serviços, características, propriedades e *handles*.	Listagem obtida por programa e confirmada no telemóvel.	1
B2	O descritor de configuração fica no *handle* seguinte ao da característica, e sem a sua escrita não chegam notificações.	Execução sem a escrita, execução com a escrita e explicação do valor escrito.	2
B3	Escrever `0x0000` no descritor interrompe as notificações sem terminar a ligação.	Registo da paragem e da retoma.	2
B4	Escrita com e sem confirmação.	Diferença observada no código e na captura de uma e de outra.	3

C. Notificação face à sondagem

Id	Resultado a demonstrar	Evidência	Nível
C1	O envio espontâneo gera menos tráfego do que a sondagem periódica para a mesma cadência.	Número de PDUs ATT em 60 s nos dois modos, contado numa captura.	3
C2	Latência de um comando: tempo entre a escrita e a chegada da resposta.	Histograma ou tabela de pelo menos 50 amostras, com mediana e máximo.	2
C3	Cadência real das notificações face ao período pedido.	Intervalos entre mensagens consecutivas para pelo menos dois períodos.	2
C4	Comando do gateway ao nó: o LED acende e a confirmação regressa ao painel.	Demonstração ao vivo e tempo medido.	1

D. Capacidade

Id	Resultado a demonstrar	Evidência	Nível
D1	Uma mensagem com mais de 20 octetos chega fragmentada em várias notificações.	Duas notificações consecutivas visíveis numa captura e relação com o MTU de 23 octetos.	3
D2	Débito útil máximo atingido entre o gateway e o nó.	Octetos úteis por segundo, com a descrição do método e do fator que limita.	3
D3	Peso do cabeçalho: octetos úteis face aos octetos totais de uma notificação.	Cálculo apoiado numa notificação capturada.	2

E. Robustez

Id	Resultado a demonstrar	Evidência	Nível
E1	Tempo que o gateway demora a dar a ligação como perdida quando o nó perde a alimentação.	Medição repetida pelo menos cinco vezes e relação com o *supervision timeout*.	2
E2	Recuperação automática: o gateway religa-se sozinho e o painel retoma.	Registo com a hora da perda, a hora da retoma e o número de mensagens perdidas pelo contador.	2
E3	Enquanto o gateway está ligado, o nó deixa de ser visível e um segundo cliente não consegue ligar-se.	Varrimento do telemóvel antes e depois, e explicação.	1
E4	Efeito da interferência na banda dos 2,4 GHz (Wi-Fi, forno de micro-ondas, corpo humano).	Perdas ou RSSI com e sem a fonte de interferência.	3

F. Observação do protocolo

Id	Resultado a demonstrar	Evidência	Nível
F1	A sequência completa de uma sessão: anúncio, pedido de ligação, troca de MTU, descoberta de serviços, escrita do descritor, notificações.	Captura com `sudo btmon` anotada, uma etapa por linha.	3
F2	O conteúdo de uma notificação em hexadecimal corresponde ao texto enviado pelo Arduino.	Octetos capturados e a sua conversão.	1
F3	Parâmetros da ligação negociados: intervalo, latência e *supervision timeout*.	Valores lidos na captura e comparação com o que a teoria prevê.	2

G. Privacidade e segurança

Id	Resultado a demonstrar	Evidência	Nível
G1	Informação recolhida sobre os dispositivos da sala sem qualquer autorização.	Tabela com endereço, tipo de endereço (público ou aleatório) e nome, e discussão.	1
G2	Qualquer cliente que se ligue ao nó pode ler os dados e comandar o LED.	Telemóvel a acender o LED sem emparelhamento e proposta de mitigação.	2

3. Demonstração

No final da sessão cada grupo mostra o sistema a funcionar e apresenta os resultados escolhidos.

1. Mostrar o sistema a funcionar, pelo menos na primeira fase.
2. Escolher pelo menos três resultados, de pelo menos duas categorias distintas, dos quais pelo menos um das categorias A, C ou F.
3. Para cada resultado preparar a evidência, que deve estar pronta a mostrar sem novas medições.

Extra

O que fazer, passo a passo

1. Montar o nó Arduino de acordo com a figura das ligações e carregar `no_ble_extra.ino`.
2. Aceder ao Raspberry Pi por SSH e confirmar que o adaptador Bluetooth está operacional (subsecção "Acesso ao Raspberry Pi").
3. Executar o varredor, identificar o endereço do nó e anotar o RSSI inicial.
4. Executar `sudo python3 codigo/ponte_ble_tcp.py <ENDERECO>` e confirmar no terminal que as notificações chegam.
5. Construir o painel Node-RED com um gráfico e confirmar que a primeira fase funciona.
6. Passar para a ponte estendida e testar os comandos, o ficheiro CSV e a religação, adaptando-a ao grupo.
7. Acrescentar ao painel um indicador do estado da ligação.
8. Percorrer as tabelas da secção 2 e escolher os resultados a demonstrar.

9. Recolher as medições e as evidências de cada resultado escolhido, com `medidor_ble.py`, `btmon` e o telemóvel.

10. Preparar a demonstração e apresentá-la ao docente.

Sem todos os sensores

O exercício precisa do HM-11 ligado ao Arduino e de mais nada. Todos os resultados da secção 2 dizem respeito à ligação BLE, e nenhum depende do DHT11 ou do LED.

No início de `no_ble_extra.ino` há duas constantes. Com `USAR_SENSOR` a 0, as leituras passam a ser simuladas por um passeio aleatório em torno de 22 °C e 45 %, sem DHT11 nem biblioteca. Com `USAR_LED` a 0, o comando `1/0` não atua sobre nenhum pino e é confirmado no Monitor Série do Arduino. O pino 13 não serve de LED alternativo, porque comanda o reset do HM-11. Os valores simulados mantêm o formato `contador,temperatura,humidade`, pelo que o painel, o ficheiro e as medições funcionam sem alterações.

Alternativas aos sensores

Se houver outros sensores, qualquer sensor ou atuador que o Arduino consiga ler ou comandar serve, desde que as leituras passem a ser enviadas pelo mesmo caminho. A alteração faz-se na função `enviar()` do sketch, que lê o sensor e escreve a mensagem na porta série do HM-11. Os pinos D2 e D3 estão ocupados pelo módulo e o pino D13 pelo seu reset. São exemplos possíveis um DHT22, um sensor analógico de temperatura, um LDR para a luminosidade, um sensor ultrassónico de distância, um sensor de movimento PIR ou um botão, e, como atuador, um zumbidor ou um módulo de relé em vez do LED.

Se mudar a grandeza medida, o painel e o ficheiro de registo devem mudar de acordo com ela. Os resultados das tabelas da secção 2 continuam a aplicar-se, porque dizem respeito à ligação BLE e não ao sensor.

Enquadramento teórico

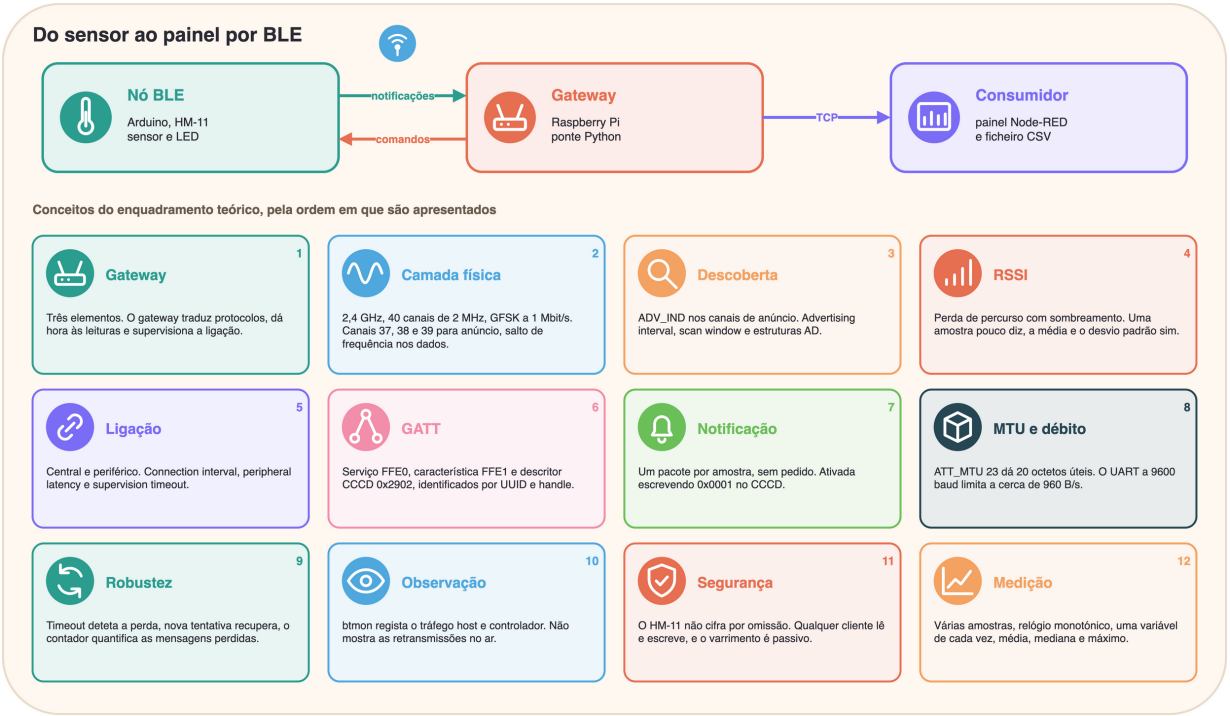


Figura 4: Resumo gráfico do enquadramento teórico

Esta secção descreve de forma sistemática o que o sistema construído faz e porque se comporta como se comporta. Cada resultado da secção 2 corresponde a uma propriedade do protocolo descrita aqui.

Arquitetura e papel do gateway

O sistema segue o modelo de três elementos habitual em Internet das Coisas. O nó de aquisição liga-se ao mundo físico, mede e atua, e dispõe de recursos mínimos de processamento, memória e energia. O gateway interliga duas redes de natureza diferente, neste caso a ligação BLE ao nó e a rede IP local. O consumidor, representado pelo painel e pelo ficheiro de registo, não conhece o BLE: recebe apenas dados já traduzidos.

O gateway desempenha quatro funções que o nó não pode assumir. Traduz protocolos, convertendo notificações GATT em mensagens de texto sobre TCP. Atribui hora de receção às leituras, porque o nó não tem relógio de tempo real. Armazena e regista, o que desacopla o ritmo do nó do ritmo do consumidor. Supervisiona a ligação e decide quando e como a restabelecer.

Camada física e acesso ao meio

O BLE funciona na banda ISM dos 2,4 GHz, dividida em 40 canais com 2 MHz de espaçamento, a uma taxa de símbolo de 1 Mbit/s com modulação GFSK. Três desses canais, o 37, o 38 e o 39, estão reservados ao anúncio e escolhidos de modo a evitar os canais mais usados pelo Wi-Fi. Os restantes 37 canais transportam os dados das ligações, com salto de frequência entre eventos de

ligação. O salto de frequência é a principal defesa do BLE contra a interferência, e explica porque é que uma fonte de interferência estreita degrada o desempenho sem o interromper.

Descoberta: anúncio e varrimento

Um periférico que aceita ligações emite periodicamente pacotes de anúncio *ADV_IND* nos três canais de anúncio. O intervalo entre anúncios, *advertising interval*, varia entre 20 ms e 10,24 s, em múltiplos de 0,625 ms, mais um pequeno atraso aleatório que reduz colisões. Um intervalo curto acelera a descoberta e aumenta o consumo; um intervalo longo faz o inverso.

Um observador em modo de varrimento escuta cada canal durante uma janela, *scan window*, repetida com um período, *scan interval*. A deteção de um anúncio depende da coincidência entre o instante e o canal da emissão e a janela de escuta. Por isso o número de deteções por unidade de tempo cresce com a frequência de anúncio e com o rácio entre a janela e o intervalo de varrimento.

Os dados de anúncio têm o formato de estruturas AD consecutivas, cada uma com um octeto de comprimento, um octeto de tipo e os dados. Os tipos mais comuns são as *flags*, o nome do dispositivo, a potência de emissão e os dados de fabricante. O endereço do dispositivo pode ser público, atribuído pelo fabricante, ou aleatório; os endereços aleatórios podem mudar periodicamente para dificultar o seguimento de uma pessoa através do seu dispositivo.

RSSI e propagação

O *Received Signal Strength Indicator* é a potência medida pelo recetor, em dBm. Num modelo de perda de percurso com sombreamento, o RSSI à distância d é

$$\mathrm{RSSI}(d) = \mathrm{RSSI}(d_0) - 10\,n \log_{10}\!\left(\frac{d}{d_0}\right) + X_\sigma,$$

em que d_0 é uma distância de referência, n o expoente de perda de percurso e X_σ uma variável aleatória de média nula e desvio padrão σ. O expoente é próximo de 2 em espaço livre e situa-se tipicamente entre 2 e 4 no interior de edifícios. O termo aleatório resume os obstáculos, a orientação das antenas, as reflexões e a presença de pessoas. Daqui decorre que uma única leitura de RSSI pouco informa sobre a distância, e que só a média de várias amostras, acompanhada do desvio padrão, tem significado estatístico.

Ligação

Para estabelecer uma ligação, o gateway, no papel de *central*, responde a um anúncio com um pedido de ligação. O nó passa a *periférico* e deixa de se anunciar. É por isso que, enquanto o gateway está ligado, o nó deixa de ser visível para outros dispositivos e não aceita um segundo cliente.

A ligação organiza-se em eventos periódicos em que central e periférico trocam pacotes. Três parâmetros governam o seu comportamento. O *connection interval*, entre 7,5 ms e 4 s, é o período entre eventos. A *peripheral latency*, entre 0 e 499, é o número de eventos que o periférico pode ignorar para poupar energia. O *supervision timeout*, entre 100 ms e 32 s, é o tempo máximo sem pacotes válidos após o qual a ligação é dada como perdida. Estes valores são propostos pelo central

e aceites ou ajustados pelo periférico, pelo que não têm de coincidir com os da documentação do módulo.

Atributos e GATT

O *Attribute Protocol* (ATT) define um servidor que expõe uma tabela de atributos, cada um com um *handle* de 16 bits, um tipo identificado por UUID e um valor. O *Generic Attribute Profile* (GATT) estrutura essa tabela numa hierarquia de serviços, características e descritores. Uma característica agrupa um valor e as propriedades que dizem como pode ser usado, por exemplo leitura, escrita, escrita sem confirmação e notificação.

O módulo HM-11 expõe um serviço de UUID `FFE0` com uma única característica `FFE1`, que se comporta como uma porta série transparente. Os octetos escritos pelo cliente saem pela porta série do módulo e os octetos que o Arduino escreve nessa porta tornam-se o valor da característica. Os *handles* dependem da implementação, e por isso o código identifica as características por UUID e deduz o descritor a partir do *handle* da característica.

Notificações e sondagem

Há dois modos de um cliente obter dados de uma característica. Na sondagem, o cliente envia um pedido de leitura e recebe uma resposta, o que gera dois pacotes ATT por amostra e fixa a latência ao período de sondagem. Na notificação, o servidor envia um pacote *Handle Value Notification* sempre que o valor muda, sem pedido prévio, o que gera um pacote por amostra e reduz a latência ao tempo de transmissão.

O cliente ativa as notificações escrevendo no *Client Characteristic Configuration Descriptor* (CCCD, UUID 0x2902) o valor 0x0001, e desativa-as escrevendo 0x0000. Sem essa escrita o servidor não envia nada, mesmo que o valor mude. A notificação não é confirmada ao nível do ATT, ao contrário da indicação. A fiabilidade é garantida abaixo, pela camada de ligação, que retransmite os pacotes não reconhecidos até ao fim do *supervision timeout*.

MTU, fragmentação e débito

O *ATT_MTU* predefinido é de 23 octetos. Descontados 3 octetos de cabeçalho ATT, cada notificação transporta no máximo 20 octetos de dados úteis. Uma mensagem maior é partida em várias notificações consecutivas, e cabe ao recetor voltar a juntá-las, usando por exemplo um delimitador de fim de linha.

O débito útil máximo é limitado por vários fatores em série. No lado BLE depende do número de pacotes por evento de ligação, do *connection interval* e dos 20 octetos por pacote. No lado do Arduino depende da porta série do módulo, que a 9600 bit/s transfere cerca de 960 octetos por segundo. O débito observado é o mínimo destes limites, o que identifica o estrangulamento do sistema. Em termos de eficiência, uma notificação de 20 octetos úteis ocupa 27 octetos de *payload* da camada de ligação, por força dos cabeçalhos L2CAP, com 4 octetos, e ATT, com 3, e acrescem 10 octetos de preâmbulo, endereço de acesso, cabeçalho e CRC. O peso do cabeçalho é por isso significativo para mensagens curtas, como as leituras de um sensor.

Robustez e recuperação

A perda de ligação é detetada pelo *supervision timeout*: o central só dá a ligação por terminada depois de esgotado este tempo sem receber pacotes do periférico. Há uma consequência prática, a de que o tempo que o gateway demora a perceber que o nó desapareceu é da ordem de grandeza desse parâmetro, e não é imediato.

A recuperação requer duas decisões de projeto. A primeira é a estratégia de nova tentativa, em que um intervalo crescente entre tentativas evita saturar o adaptador quando o nó está indisponível. A segunda é a deteção de perdas durante a indisponibilidade, que se resolve numerando as mensagens no nó. Como o contador é monotónico, uma falha na sequência recebida quantifica exatamente as mensagens perdidas.

Observação do protocolo

A ferramenta `btmon` regista o tráfego na interface entre o *host*, que corre no Raspberry Pi, e o controlador Bluetooth. Mostra por isso os eventos de anúncio recebidos, o pedido e a conclusão da ligação com os seus parâmetros, as trocas de MTU, a descoberta de serviços, a escrita do CCCD e cada notificação com o seu conteúdo em hexadecimal. Não mostra o que se passa no ar, como as retransmissões da camada de ligação, que são resolvidas dentro do controlador. Uma captura do ar exige um *sniffer* dedicado, que não está disponível neste exercício.

Segurança e privacidade

O BLE prevê emparelhamento, cifra da ligação e autenticação, mas o HM-11 usa por omissão uma ligação sem proteção. Qualquer cliente dentro do alcance pode ligar-se, ler os dados e escrever na característica, o que inclui comandar o LED. O varrimento é passivo e não requer autorização, pelo que a lista de dispositivos de uma sala, com endereços e nomes, é recolhida por qualquer observador. Endereços aleatórios que mudam ao longo do tempo reduzem o seguimento, mas só os dispositivos que os usam beneficiam dessa proteção.

Metodologia de medição

Os resultados do exercício são medições, e o seu valor depende do método. Uma medição tem de indicar o que foi medido, em que condições, com que instrumento e com quantas repetições.

Para grandezas aleatórias como o RSSI e a latência, uma única amostra é insuficiente. Recolhe-se um número de amostras suficiente para estimar a média e o desvio padrão, ou a mediana e os percentis quando a distribuição é assimétrica, como acontece quase sempre com latências. O valor máximo é relevante porque num sistema de tempo real é a cauda da distribuição que determina o comportamento no pior caso.

Os intervalos de tempo medem-se com um relógio monotónico, que não sofre os saltos do relógio do sistema. A latência de um comando é a diferença entre o instante em que a escrita é emitida e o instante em que a resposta chega, e inclui por isso o tempo de espera pelo próximo evento de ligação, o processamento no Arduino e a transmissão pela porta série do módulo. Convém alterar

uma variável de cada vez, mantendo a posição, a orientação e o ambiente constantes, para que a diferença observada se possa atribuir a essa variável.

Relação com os resultados da secção 2

As categorias de resultados correspondem aos tópicos acima. A categoria A decorre do anúncio, do varrimento e do modelo de propagação. A categoria B decorre da estrutura GATT e do CCCD. A categoria C compara o modelo de sondagem com o de notificação. A categoria D decorre do MTU, da fragmentação e dos limites de débito. A categoria E decorre do *supervision timeout*, da ligação exclusiva e da estratégia de recuperação. A categoria F verifica os mesmos conceitos na captura do tráfego. A categoria G decorre das propriedades de segurança e de privacidade do protocolo.

Questões para discussão

1. Que funções desempenha o gateway que o periférico não poderia desempenhar, e porquê?
2. Se houvesse dez nós em vez de um, que alterações seriam necessárias na ponte?
3. Porque é que a ligação TCP local usada neste exercício não serviria numa instalação distribuída por vários edifícios?
4. Que informação sobre os dispositivos da sala foi possível recolher sem qualquer autorização prévia?
5. Quanto tempo decorre até a ponte dar a ligação como perdida quando o nó perde a alimentação, e com que parâmetro da ligação se relaciona esse valor?

Referências

- Documentação da biblioteca bluepy: `https://ianharvey.github.io/bluepy-doc/`
- Raspberry Pi Foundation. Configuração *headless* e acesso por SSH: `https://www.raspberrypi.com/documentation/`
- Documentação do Node-RED: `https://nodered.org/docs/user-guide/`
- Documentação do BlueZ e da ferramenta `btmon`: `http://www.bluez.org/`
- Folha de dados do módulo HM-11, com a lista de comandos AT: `http://www.jnhuamao.cn/`